

# Blocking is an insufficient description for senders and receivers

## Abstract

Proposes an initial direction for a reformulation and extension of the blocking property to senders. No wording is suggested as of this revision.

## Introduction

The most recent revision of [P0443, A Unified Executors Proposal for C++](#) specifies a number of generic properties, the vast majority of which are focused on the particulars of executor types and, secondarily, schedulers. Recent design work, however, has emphasized the importance of senders, receivers, and schedulers to the overall picture. There is as of late an increased understanding that these concepts likely represent the fundamental abstractions for generic concurrent programming, rather than executors. Indeed, eager executors should probably be viewed as limited tools of expedience rather than fundamental abstractions. For reference see the papers [One-Way is a Poor Basis Operation](#), and [Disentangling schedulers and executors](#).

Even so, most aspects of the design work laid out already for executors remains important for senders and receivers. In particular are the properties of certain classes of types, with which generic algorithms may conditionally enabled and optimized.

New approaches to specifying these properties have been suggested in [redefine properties in P0443](#), the general direction of which we agree with and the proposed mechanism of which we'll assume for the rest of this paper.

Our concern for the remainder of this paper shall be the blocking property, which [P0443](#) specifies as "[describing] what guarantees executors provide about the blocking behavior of their execution functions." When adapted to a property query of sender types, we can use this information to perform library-internal optimizations. For instance, it can be shown that a default implementation of the `execution::submit()` algorithm for senders and receivers can elide heap allocations, conditional on whether the given sender guarantees it blocks execution of the calling thread pending completion of the operation.

This alone should be sufficient motivation to redesign the properties in [P0443](#) to apply generically to types other than executors, but there are other benefits as well. Staying focused on the blocking property, this allows for more ergonomic and streamlined implementations of custom sender types, which *could* choose to omit customization of `submit()` entirely, provided a guarantee that the default implementation in terms of `connect()` and `start()` will be just as efficient. As it stands now, custom sender types, even those that can guarantee completion inline such as the proposed algorithm `just()` from [P1897](#), would likely always have to customize `submit()` to avoid unnecessary heap allocations.

A straightforward adaptation of the blocking property to senders appears to not be possible, however, and [issue #480 of the executors design review](#) from earlier this year highlighted the basic problems. We believe some of

the issues with this can be resolved by carefully reformulating the blocking property.

## Proposal

First, we believe that blocking is the wrong description for how senders behave, at least to a point. The principal benefit of senders and receivers is their ability to compose cleanly, efficiently, and *lazily*. Describing a sender with terms relating to blocking, therefore, seems inappropriate. Doubly so since senders are just *one half* of the picture. We think blocking should instead be reserved for describing operations, which we will explore in more detail later on.

As regards the potential for eliding heap allocations in `execution::submit()`, what matters is not blocking *per se*, but rather *how and when* a sender completes, and in *which context* a connected receiver's completion channels are guaranteed to be signaled. In particular, a default implementation of `execution::submit()` may elide allocation of temporary state and enjoy the efficiency of a direct implementation along the lines of

```
operation_state auto op = execution::connect(S, R);
execution::start(op);
```

if and only if the sender type `S` can guarantee it fulfills the receiver contract synchronously with the invocation of `start()`. This likely means it must guarantee a *strongly happens before* relationship with return from `start()`. Notice how this is *not* a description of a blocking operation in the general case, but rather a description of a synchronous operation, and we believe it would be unfortunate to conflate those two terms.

Consider for instance the algorithm `just()` mentioned previously. This is probably not what most would consider a "blocking" algorithm -- in fact, that description is in disagreement with the standard's definition of "blocking" -- but the only language provided by [P0443](#) to describe it would be as "always blocking". The same goes for inline schedulers and inline executors.

As a consequence, we think senders should first and foremost be described by their *completion guarantees*. Taking the language of the blocking property and turning it around, more or less, we'd have the following possibilities for guarantees a sender type might make:

- `unspecified_completion_t`, from the prior description `possibly_blocking_t`, guaranteeing nothing about when or where a connected receiver's completion- signal operations may occur;
- `asynchronous_completion_t`, from the prior description `never_blocking_t`, guaranteeing that a connected receiver's completion-signal operations will not occur on the calling thread before `execution::start()` returns, but does not prohibit them from occurring concurrently on another thread prior to, concurrently with, or after return from `execution::start()`;
- and `synchronous_completion_t`, from the prior description `always_blocking_t`, guaranteeing that a connected receiver's completion- signal operations will occur before `execution::start()` returns, but does not guarantee on which thread an operation may occur -- specifically, they need not occur on the thread calling `execution::start()`.

We can also strengthen the requirement for `synchronous_completion_t` to obtain another possibly useful guarantee:

- `inlined_completion_t`, guaranteeing that a connected receiver's completion- signal operations will occur before `execution::start()` returns, and on the thread calling `execution::start()`.

The default assumption in generic code would be `unspecified_completion_t` when interfacing with a sender type that does not customize this property.

Now, we mentioned earlier how blocking should be a description of *operations* rather than senders, so we suggest something like the blocking property be redesigned to describe operation states.

Before moving on, recall the definition of blocking provided in [defns.block](#)

(execution) wait for some condition (other than for the implementation to execute the execution steps of the thread of execution) to be satisfied before continuing execution past the blocking operation

Note that we do not think this should be a property of senders, because senders *do not* comprise the whole picture of an asynchronous operation, and *do not* have visibility into the work performed underneath a call to `execution::set_value()`. Likewise, receivers, which represent the completions of (possibly intermediate) asynchronous operations, have no visibility into the upstream computations of the senders they are connected to. Both of these facts are a *good thing* for the design! But it does mean that both halves matter equally when determining whether a fully composed operation is blocking.

This is all to say, in the general case we do not know whether an operation is blocking until both sender and receiver are connected. Indeed, we believe it is necessary for information about blocking to back-propagate from receiver to sender at the time of connecting one to the other, and then forward again to any code requiring knowledge of it, exposed through the returned operation state. This is appropriate since a given thread of execution ought to care mostly about the behavior of `execution::start()`. Also, the required property queries could be performed with the same customization point, and we suggest the name `get_blocking` from [P2220](#) is retained for that purpose.

Moreover, the above description intuitively corresponds to the distinction between schedulers, composed lazily with senders and receivers, and executors, used eagerly. The language of blocking, appropriately redefined to describe operations, could even be used to recover the description of blocking for executors.

We also have the following rough descriptions for blocking properties redefined for operation states, adapted from the wording in [P2220](#):

- `possibly_blocking_t`, guaranteeing nothing about invocation of an operation state's `start()` customization, execution may block pending some condition external to the steps of the thread of execution when invoking `execution::start()`;
- `never_blocking_t`, guaranteeing that execution shall not block pending any condition external to the steps of the thread of execution when invoking `execution::start()`;
- and `always_blocking_t`, guaranteeing that execution shall block pending some condition external to the steps of the steps of the thread of execution when invoking `execution::start()`;

Like that for the completion property, the default assumption in generic code would be `possibly_blocking_t` when interfacing with an operation that does not customize this property.

Disregarding for the moment the categories `unspecified_completion_t` and `possibly_blocking_t`, we think there are four meaningful combinations of completion guarantees for senders and blocking guarantees of the operation states they produce when connected to a receiver. These are, with brief concrete examples of each:

- `asynchronous_completion_t` / `never_blocking_t`, such as work enqueued to a background thread pool that completes in a resident thread (assuming the enqueue performed in `start()` can be implemented in a non-blocking manner);
- `synchronous_completion_t` / `always_blocking_t`, such as fork/join parallelism synchronously waited on for completion (consider `bulk_schedule` to a thread pool or GPU resource followed by `sync_wait()`);
- `inlined_completion_t` / `always_blocking_t`, such as a write to or read from a network socket configured in blocking mode, resulting in the number of bytes transferred with no additional dependent work;
- and `inlined_completion_t` / `never_blocking_t`, such as a write to or read from a network socket configured in non-blocking mode, resulting in the number of bytes transferred with no additional

dependent work.

Note how the language of blocking as currently specified for [P0443](#) is insufficient to distinguish between all of the above examples.

It is worth emphasizing the last two examples above. When applying senders and receivers to future designs of fundamental I/O abstractions, the blocking property allows us to express that asynchrony is not required to guarantee non-blocking operation. It would be unfortunate to not have the requisite vocabulary to describe this fact.

## Impact

One significant benefit obtained with the above design is a disentangling of the concerns around blocking operations and completion guarantees. They are properly orthogonal, and our suggested direction reflects this. We also think this is a direction that's more harmonious with the language's current descriptions of blocking and concurrency. Moreover, when applying senders and receivers to the implementation of latency sensitive and safety critical applications, it may be paramount to afford generic code comprising execution contexts and runtime systems a deep understanding of the work being scheduled and executed. And by reflecting these properties in the primitive layers of a design, in a way that's consistent with their fundamental mode of operation, can let us achieve this.

## Limitations and Open Questions

It is still unclear how forward progress guarantees (specifically, the concurrent, parallel, and weakly parallel guarantees described in the standard) fit into this picture. We believe further research is needed in this direction, along with an appropriate description for execution allowances along the lines of sequenced, parallel, parallel-unsequenced, and unsequenced.

The completion properties for senders described above are assuming each of the receiver completion channels are of equal status. This may not be appropriate, and it could be desirable to instead focus on the value channel, specifically, allowing senders wide discretion in choosing when and where the error and/or cancellation channels are signaled, without compromising their completion guarantee. For example, can an operation that initiates a truly asynchronous computation still claim "completes asynchronously" if sometimes it must call `execution::set_error()` or `execution::set_done()` on the initiating thread? We believe so, but the wording will have to be specified carefully to avoid confusion.